Ziplofy – Best Practices Audit

Version 1.1 | February 10, 2025 | Admin Frontend + Ziplofy3b Backend

> **What this document is**
> We list suggested improvements for our codebase, with **real companies that use each practice** and where we can learn more.

Contents

1. Quick Summary

What we're missing (priority fixes)

- Rate limiting on login/OTP
- Security headers (Helmet)
- Input validation library
- Skeleton loaders instead of plain "Loading..."
- Profile self-service (edit name, change password, profile picture)
- Code splitting (React.lazy) for faster load
- .env.example and project README

What we're doing well

- CORS, password hashing (bcrypt), error middleware, async error handling

2. Suggested Changes & Companies Using Them

Security

Rate limiting on auth endpoints

We would limit login/OTP attempts to prevent brute force.

Used by: Stripe, Cloudflare, GitHub, AWS, Twilio

Reference: OWASP DoS Cheat Sheet, express-rate-limit on npm

Helmet – HTTP security headers

We would add X-Frame-Options, X-Content-Type-Options, etc.

Standard for Express apps; recommended by Express docs

Reference: expressjs.com/advanced/best-practice-security, helmetjs.github.io

Input validation library (express-validator, Joi, Zod)

We would validate and sanitize request body, query, params.

express-validator: 1.3M+ weekly npm downloads; used across Node.js ecosystem

Reference: OWASP Input Validation, express-validator.github.io

Frontend / UI

Skeleton loaders

We would show gray placeholder shapes while content loads, instead of "Loading...".

LinkedIn, Facebook, YouTube, Slack, Uber, eBay, Headspace

Reference: Nielsen Norman Group – Skeleton Screens

Profile self-service

We would let our users edit name, change password, upload profile picture.

Google, Microsoft, GitHub, Slack, Notion – standard for any product with accounts

Reference: OWASP Password Storage, common UX patterns

Consistent error display

We would use one pattern (e.g. toast or inline) instead of our current mix of alert(), toast, inline.

Stripe, Vercel, Linear – consistent toast/inline errors

Reference: Nielsen Norman Group – Error Message Guidelines

Accessibility

ARIA labels & focus management in modals

We would enable screen readers and keyboard users to use our UI properly.

Microsoft, Google, Apple – publish WCAG conformance reports; required for gov/enterprise

Reference: W3C ARIA Authoring Practices, w3.org/WAI/ARIA/apg

Performance

Code splitting (React.lazy + Suspense)

We would load only the code needed for the current page.

Facebook (facebook.com redesign), Netflix, Airbnb – documented use of lazy loading

Reference: react.dev/reference/react/lazy, web.dev/code-splitting-suspense

Developer Experience

.env.example + project README

We would add a template for env vars and setup instructions.

Heroku (12-Factor App creator), Vercel, Docker – standard for deployable apps

Reference: 12factor.net/config

3. What We're Already Doing Well

- **CORS** – we have it configured with origin and credentials
- **Password hashing** – we use bcrypt with salt in our User model
- **Centralized error handling** – we use CustomError + error middleware

- **Async error wrapper** – we use asyncErrorHandler
- **Empty states** – e.g. we show them on the ExportLogs page

4. Reference Websites

- OWASP – cheatsheetseries.owasp.org
- W3C ARIA – w3.org/WAI/ARIA/apg
- Nielsen Norman Group (UX) – nngroup.com
- WCAG – w3.org/WAI/WCAG21/quickref
- web.dev – web.dev
- React – react.dev
- Express Security – expressjs.com/en/advanced/best-practice-security
- 12-Factor App – 12factor.net
- express-validator – express-validator.github.io
- express-rate-limit – github.com/nfriedly/express-rate-limit
- Helmet – helmetjs.github.io
